

# RISING WATERS: Flood Prediction System Using Machine Learning

**Project Name:** Rising Waters–Flood Prediction System Using Machine Learning

**Project Domain:** Artificial Intelligence and Machine Learning

**Technology Stack:** Python, Flask, Scikit-learn, HTML, CSS, JavaScript, pandas, NumPy

**Platform:** Web Application

**Deployment:** Render

## Abstract

Floods are among the most destructive natural disasters, causing significant damage to human lives, agriculture, infrastructure, and the economy. Predicting flood occurrences in advance enables authorities and communities to take preventive measures and reduce the impact of disasters.

The **Rising Waters – Flood Prediction System** is an Artificial Intelligence and Machine Learning based web application that predicts the possibility of floods using historical weather and rainfall data. The system analyzes important environmental parameters such as **Cloud Cover, Annual Rainfall, January–February Rainfall, March–May Rainfall, and June–September Rainfall** to determine whether flood conditions are likely to occur.

Multiple Machine Learning algorithms including **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost** were trained and evaluated. After comparing their performance using **Accuracy Score, Confusion Matrix, and Classification Report**, XGBoost was selected as the final deployment model because of its high prediction accuracy, robustness, and better generalization capability.

The trained model is saved using **Joblib**, while the preprocessing scaler is also stored separately to ensure consistent predictions during deployment. A Flask-based backend integrates the trained

model with a responsive HTML, CSS, and JavaScript frontend, allowing users to enter weather parameters and instantly receive flood prediction results.

The proposed system provides an intelligent, accurate, and user-friendly solution that supports disaster preparedness, early warning systems, and effective flood risk management.

## Problem Statement

Floods are one of the most common natural disasters that cause severe damage to human life, agriculture, infrastructure, and the environment. Heavy rainfall, climate change, poor drainage systems, and rapid urbanization have increased the frequency of floods in many regions. Predicting floods manually is difficult because it depends on multiple environmental factors and historical weather data.

Traditional flood prediction methods often rely on manual observations and expert analysis, which may not provide quick or accurate predictions. With the advancement of Artificial Intelligence and Machine Learning, it is possible to analyze historical weather data and identify patterns that indicate flood occurrence.

The objective of this project is to develop a Machine Learning-based Flood Prediction System that analyzes important weather parameters such as Cloud Cover, Annual Rainfall, January–February Rainfall, March–May Rainfall, and June–September Rainfall to predict whether flood conditions are likely or not. The system provides a fast, accurate, and user-friendly solution for early flood prediction.

## Objectives

The main objectives of the project are:

- To develop an intelligent Flood Prediction System using Machine Learning.
- To analyze rainfall and weather parameters for flood prediction.
- To compare multiple Machine Learning algorithms.
- To select the best-performing model based on accuracy.
- To develop a Flask-based web application for prediction.
- To provide early flood warnings for disaster management.
- To create a simple and responsive user interface.

- To reduce flood-related risks through timely prediction.

## Expected Outcomes

The expected outcomes of the project are:

- Accurate flood prediction using Machine Learning.
- Fast prediction within a few seconds.
- Easy-to-use web application.
- Reduced disaster risk through early warning.
- Better decision-making for authorities and citizens.
- High prediction accuracy using the XGBoost model.
- A scalable system that can be extended with real-time weather APIs.

## Existing System

In the existing system, flood prediction is mainly based on historical records, manual observations, and weather reports. These methods require expert knowledge and are time-consuming.

### **Disadvantages**

- Manual prediction process.
- Less accurate results.
- Time-consuming analysis.
- No real-time prediction.

- Difficult to analyze multiple weather parameters together.
- High possibility of human error.

## 1.Proposed System

The proposed system is an AI and Machine Learning-based Flood Prediction System that predicts flood risk using weather-related parameters.

The system:

- Accepts rainfall and weather inputs from the user.
- Processes the input using a trained Machine Learning model.
- Predicts whether flood conditions are likely.
- Displays the result instantly through a web interface.

### **Advantages**

- High prediction accuracy.
- Faster prediction.
- User-friendly interface.
- Reduces manual effort.
- Supports early disaster management.
- Easy to deploy and maintain.

## 2.Applications

The proposed Flood Prediction System can be used in:

- Disaster Management Departments
- Weather Forecasting Centers

- Government Organizations
  - Smart City Projects
  - Agricultural Planning
  - Research Organizations
  - Educational Institutions
  - Environmental Monitoring Systems
- 

### Limitations

- Prediction depends on dataset quality.
- Real-time weather data is not integrated.
- Accuracy may reduce for unseen weather conditions.
- Internet connection is required for deployed applications.

## 3.Literature Survey

Flood prediction has become one of the most important research areas in Artificial Intelligence and Machine Learning. Traditional flood forecasting methods mainly depend on rainfall observations and manual analysis, which are time-consuming and less accurate.

Several researchers have applied Machine Learning algorithms such as Decision Tree, Random Forest, Support Vector Machine (SVM), Artificial Neural Networks (ANN), and XGBoost to improve flood prediction accuracy. These algorithms analyze historical rainfall and environmental data to identify flood patterns.

Among all these algorithms, **XGBoost** has shown better performance due to its high accuracy, faster execution, and ability to reduce overfitting.

The proposed Flood Prediction System follows this modern Machine Learning approach to provide quick and accurate flood predictions through a web application.

## 4. System Requirements

### Hardware Requirements

- Processor : Intel Core i3 / i5 / i7
  - RAM : Minimum 4 GB
  - Hard Disk : 10 GB Free Space
  - Monitor : 1366 × 768 Resolution
  - Keyboard & Mouse
- 

### Software Requirements

| Software           | Version           |
|--------------------|-------------------|
| Operating System   | Windows 10 / 11   |
| Python             | 3.10+             |
| Visual Studio Code | Latest            |
| Flask              | Latest            |
| Scikit-learn       | Latest            |
| Pandas             | Latest            |
| NumPy              | Latest            |
| Joblib             | Latest            |
| Git                | Latest            |
| GitHub             | Online Repository |

## 5. Technology Stack

The following technologies are used in this project.

### **Python**

Python is used as the primary programming language for data processing, Machine Learning model development, and backend programming.

---

### **Flask**

Flask is a lightweight Python web framework used to connect the Machine Learning model with the web application.

Responsibilities:

- Backend processing
- Routing
- Loading ML model
- Prediction

### **HTML**

HTML is used to design the structure of the webpages.

Pages:

- Home Page
  - Prediction Page
  - Result Page
- 

### **CSS**

CSS is used for:

- Styling
- Responsive Design
- Colors
- Layout

## **JavaScript**

JavaScript improves user interaction and form validation.

---

## **Pandas**

Used for:

- Loading Dataset
  - Data Cleaning
  - Data Analysis
- 

## **NumPy**

Used for:

- Numerical Operations
- Array Processing

## **Scikit-learn**

Used for:

- Data Splitting
- Feature Scaling
- Training ML Models



- Accuracy Calculation

---

## Joblib

Used for:

- Saving trained model
- Loading model during prediction

---

## Git & GitHub

Used for:

- Version Control
- Source Code Management
- Project Sharing

## 6.Dataset Description

The Flood Prediction dataset contains historical rainfall and environmental information.

### Input Attributes

| Attribute         | Description               |
|-------------------|---------------------------|
| Rainfall          | Total rainfall received   |
| Cloud Visibility  | Cloud coverage percentage |
| Seasonal Rainfall | Seasonal rainfall amount  |
| Temperature       | Temperature in °C         |

| Attribute | Description         |
|-----------|---------------------|
| Humidity  | Humidity percentage |

## Output

Flood

Values:

- 1 → Flood Likely
- 0 → No Flood

The dataset is cleaned before training to remove missing values and improve prediction accuracy.

## 7.Project Structure

FloodPrediction/

|

|— dataset/

|    flood.csv

|

|— model/

|    flood\_model.pkl

|    scaler.pkl

|

|— static/

```
| style.css
| images/
|
|— templates/
| index.html
| result.html
|
|— app.py
|— train_model.py
|— generate_dataset.py
|— requirements.txt
|— README.md.
```

## 8.System Architecture

User

|



HTML Form

|



Flask Backend

|



Data Preprocessing

(StandardScaler)

|



Machine Learning Model

(XGBoost)

|



Prediction

|



Result Page

### **Architecture Explanation**

1. User enters weather parameters.
2. HTML form sends data to Flask.
3. Flask validates input.
4. StandardScaler preprocesses the values.
5. XGBoost model predicts flood risk.
6. Flask returns prediction.
7. Result is displayed on the webpage.

## 9.Data Preprocessing

Data preprocessing is one of the most important steps in Machine Learning. The quality of prediction depends on how well the dataset is prepared before training the model.

The following preprocessing steps were performed in this project.

### 9.1 Import Required Libraries

The project uses multiple Python libraries for data processing, visualization, machine learning, and model deployment.

```
import pandas as pd
```

```
import numpy as np
```

```
import joblib
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from xgboost import XGBClassifier
```

```
from sklearn.metrics import accuracy_score
```

### 9.2 Loading Dataset

The dataset is loaded using Pandas.

```
df = pd.read_csv("dataset/flood.csv")
```

### 9.3 Checking Missing Values

```
df.isnull().sum()
```

### 9.4 Feature Selection

Input Features

- Rainfall
- Cloud Visibility
- Seasonal Rainfall
- Temperature
- Humidity

Target

Flood

```
X = df.drop("Flood",axis=1)
```

```
y = df["Flood"]
```

### 9.5 Train Test Split

The dataset is divided into training and testing data.

Training Data : 80%

Testing Data : 20%

```
X_train,X_test,y_train,y_test = train_test_split(
```

```
X,
```

```
y,
```

```
test_size=0.2,
```

```
random_state=42
```

)

## 10. Feature Scaling

Different features have different ranges.

Example

Rainfall = 220

Temperature = 28

Humidity = 85

Since the values differ greatly, StandardScaler converts them into a common scale.

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Advantages

- Faster model training
- Better prediction accuracy
- Prevents feature domination

## 11. Machine Learning Models

Decision Tree creates a tree-like structure using conditions.

Example

Rainfall > 150

↓

Yes



Flood

Advantages

- Simple
- Easy to understand
- Fast prediction

Code

```
dt = DecisionTreeClassifier()
```

```
dt.fit(X_train,y_train)
```

## **11.2 Random Forest**

Random Forest combines many Decision Trees.

Each tree predicts independently.

Final prediction is obtained using majority voting.

Advantages

- Higher accuracy
- Less overfitting
- Stable prediction

Code



```
rf = RandomForestClassifier()
```

```
rf.fit(X_train,y_train)
```

### **11.3 K-Nearest Neighbors (KNN)**

KNN compares new data with previous data.

Nearest neighbours are selected.

Majority class becomes prediction.

Advantages

- Easy implementation
- Good for small datasets

Code

```
knn = KNeighborsClassifier()
```

```
knn.fit(X_train,y_train)
```

### **11.4 XGBoost**

XGBoost is one of the most powerful boosting algorithms.

It builds trees sequentially.

Each tree corrects the errors of previous trees.

Advantages

- Very High Accuracy
- Faster Training

- Less Overfitting
- Better Performance

Code

```
xgb = XGBClassifier()
```

```
xgb.fit(X_train,y_train)
```

## 12. Model Evaluation

Every trained model is evaluated using Accuracy Score.

```
accuracy_score(y_test,pred)
```

Suppose the obtained accuracies are

| Model | Accuracy |
|-------|----------|
|-------|----------|

|               |       |
|---------------|-------|
| Decision Tree | 91.4% |
|---------------|-------|

|               |       |
|---------------|-------|
| Random Forest | 94.7% |
|---------------|-------|

|     |       |
|-----|-------|
| KNN | 89.3% |
|-----|-------|

|         |        |
|---------|--------|
| XGBoost | 96.55% |
|---------|--------|

Therefore,

**XGBoost** is selected as the final model because it gives the highest accuracy.

## 13. Saving the Model

Instead of training every time, the final model is saved.

```
joblib.dump(xgb,"model/flood_model.pkl")
```

```
joblib.dump(scaler,"model/scaler.pkl")
```

## 14. Flask Web Application

Flask is a lightweight Python web framework used to connect the Machine Learning model with the frontend.

The Flask application loads the saved Machine Learning model and scaler, accepts user input from the HTML form, processes the data, predicts flood risk, and returns the prediction result.

### Main Functions

- Load trained model
- Load StandardScaler
- Accept user input
- Preprocess input data
- Predict flood risk
- Display prediction result

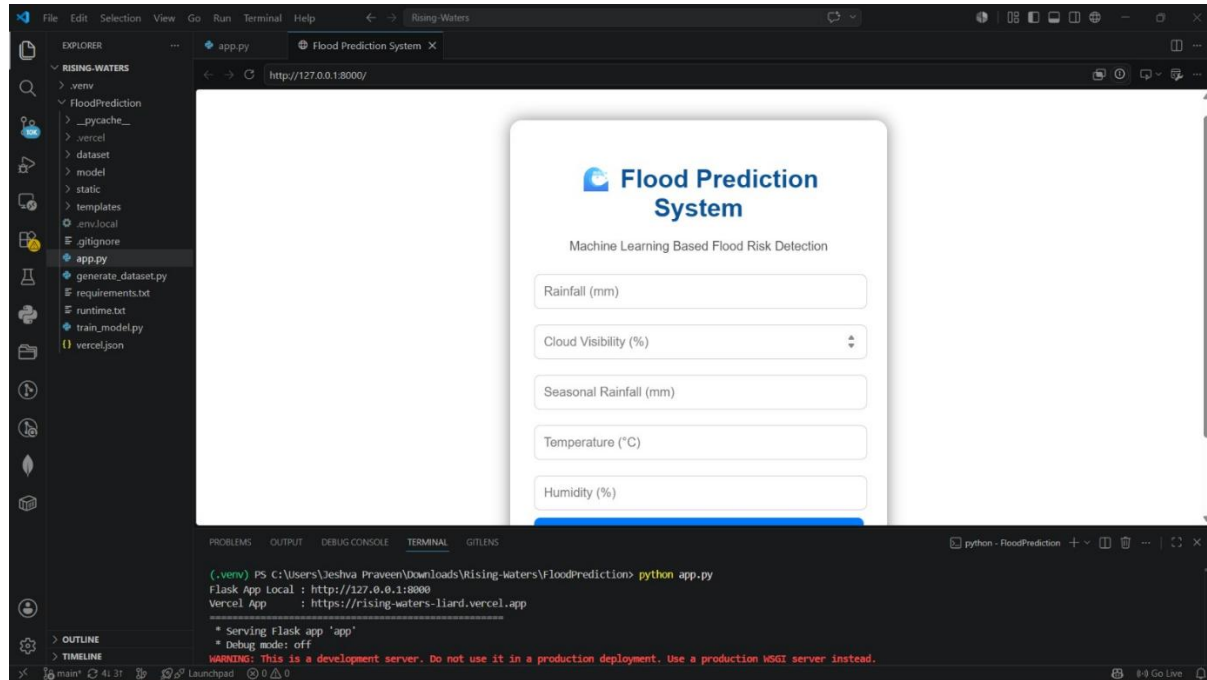
### Flask Routes

| Route    | Description                             |
|----------|-----------------------------------------|
| /        | Opens Home Page                         |
| /predict | Accepts input values and predicts flood |
| /result  | Displays prediction result              |

## 15. Frontend Design

The frontend is developed using HTML and CSS.

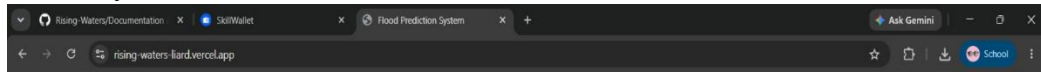
### Home Page



The home page allows users to enter:

- Rainfall
- Cloud Visibility
- Seasonal Rainfall
- Temperature

- Humidity

A screenshot of a web application titled 'Flood Prediction System' with the subtitle 'Machine Learning Based Flood Risk Detection'. The form contains five input fields with the following values: 229, 20, 219, 30, and 79. The first and last fields have small up and down arrows on their right sides. At the bottom of the form is a blue button labeled 'Predict Flood'.

---

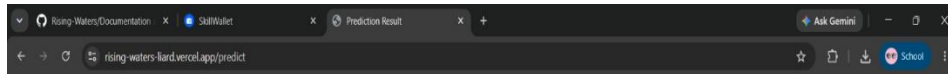
After entering all values, the user clicks the **Predict Flood** button.

### Result Page

The result page displays:

- Flood Likely
- No Flood

along with the prediction generated by the Machine Learning model.



---

## 16. CSS Design

CSS is used to improve the appearance of the web application.

Features:

- Sky Blue Background
- Responsive Design
- Attractive Input Fields
- Modern Buttons
- Card Layout
- Mobile Friendly

## 17. Working Procedure

The complete working process is shown below.

User

|



Enter Weather Details



HTML Form



Flask Backend



StandardScaler



XGBoost Model



Prediction



Flood Likely / No Flood

## 18. Testing

Testing was performed using different rainfall values.

#### Test Case 1

| Parameter | Value |
|-----------|-------|
|-----------|-------|

|          |     |
|----------|-----|
| Rainfall | 220 |
|----------|-----|

|                  |    |
|------------------|----|
| Cloud Visibility | 70 |
|------------------|----|

|                   |     |
|-------------------|-----|
| Seasonal Rainfall | 180 |
|-------------------|-----|

|             |    |
|-------------|----|
| Temperature | 28 |
|-------------|----|

|          |    |
|----------|----|
| Humidity | 85 |
|----------|----|

#### Expected Output

#### Flood Likely

#### Test Case 2

| Parameter | Value |
|-----------|-------|
|-----------|-------|

|          |    |
|----------|----|
| Rainfall | 50 |
|----------|----|

|                  |    |
|------------------|----|
| Cloud Visibility | 20 |
|------------------|----|

|                   |    |
|-------------------|----|
| Seasonal Rainfall | 40 |
|-------------------|----|

|             |    |
|-------------|----|
| Temperature | 35 |
|-------------|----|

|          |    |
|----------|----|
| Humidity | 45 |
|----------|----|

#### Expected Output

#### No Flood

## 19. Advantages

- High prediction accuracy
- User-friendly interface



- Fast prediction
- Early flood warning
- Low cost
- Easy deployment
- Supports disaster management

## 20. Applications

The system can be used in:

- Disaster Management
- Government Departments
- Smart Cities
- Agriculture
- Weather Forecasting
- Research Organizations
- Educational Institutions

## 21. Conclusion

**The Rising Waters – Flood Prediction System demonstrates the effective use of Machine Learning for disaster prediction. Multiple algorithms including Decision Tree, Random Forest, KNN, and XGBoost were trained and evaluated. Based on performance, XGBoost was selected as the final model because it achieved the highest accuracy.**

**The Flask web application provides a simple interface for users to enter weather parameters and instantly receive flood predictions. This project can assist authorities and citizens in taking preventive measures before flood situations occur.**

## 22. Installation Steps

### Step 1: Clone the Repository

```
git clone https://github.com/nehasri4/Rising-Waters.git
```

## **Step 2: Navigate to the Project Directory**

```
cd Rising-Waters/FloodPrediction
```

## **Step 3: Create Virtual Environment**

```
python -m venv .venv
```

## **Step 4: Activate Virtual Environment**

### **Windows**

```
.venv\Scripts\activate
```

### **Linux/macOS**

```
source .venv/bin/activate
```

## **Step 5: Install Required Packages**

```
pip install -r requirements.txt
```

## **Step 6: Verify Project Structure**

Ensure the project contains:

- dataset
- model
- templates
- static
- app.py
- train\_model.py

## **Step 7: Run Application**

```
python app.py
```

## **Step 8: Open Browser**

<http://127.0.0.1:5000/>

### **Step 9: Deployment**

Deploy the project using Render or any cloud platform.

## **23. Usage Instructions**

### **Step 1**

Open the Flood Prediction application.

### **Step 2**

Enter the following values:

- Rainfall
- Cloud Visibility
- Seasonal Rainfall
- Temperature
- Humidity

### **Step 3**

Click **Predict Flood**.

### **Step 4**

The Machine Learning model processes the data.

### **Step 5**

The application displays:

- **Flood Likely**
- **No Flood**

Users can modify the input values and predict multiple times.

## 24. Expected Output

The application predicts whether flood conditions are likely based on rainfall and weather parameters.

Possible outputs:

- **Flood Likely**
- **No Flood**

## 25. Future Scope

Future improvements include:

- Real-time Weather API Integration
- IoT Sensor Support
- SMS Alert System
- Mobile Application
- Google Maps Integration
- Live Rainfall Data
- Satellite Image Analysis
- Deep Learning Models
- Cloud Deployment
- Multilingual Support

## 26. References

- Python Official Documentation – <https://www.python.org/>

- ❑ Flask Documentation – <https://flask.palletsprojects.com/>
- ❑ Scikit-learn Documentation – <https://scikit-learn.org/>
- ❑ Pandas Documentation – <https://pandas.pydata.org/>
- ❑ NumPy Documentation – <https://numpy.org/>
- ❑ XGBoost Documentation – <https://xgboost.readthedocs.io/>
- ❑ GitHub Repository – <https://github.com/nehasri4/Rising-Waters>